

Document Tehnic - Etapa 1

Remailer

Sistem de Gestiune si Distributie Asincrona a Notificarilor

Echipa de Proiect:

Lemnaru Mihai-Daniel (Grupa 344C4)

Peter Arthur-Cristian (Grupa 344C4)

Responsabil Laborator:

Alexandra-Ana-Maria ION

1 Prezentare Generala si Tehnologii Folosite

Scopul proiectului este dezvoltarea intregului ciclu de viata al unei aplicatii cloud-native de tip „*Notification-as-a-Service*”, capabila sa preia, sa programeze si sa trimita notificari (e-mails) intr-un mod scalabil si asincron.

Sistemul este construit pe baza unei arhitecturi de tip microservicii, fiind impartit in mai multe componente independente. Pentru gestionarea si coordonarea acestora, aplicatia utilizeaza **Kubernetes**, ceea ce ajuta la obtinerea unei aplicatii scalabile, stabile si bine izolata la nivel de retea. Procesul de deployment este automatizat printr-un pipeline de tip **CI/CD** (GitHub Actions), care construiesc si publica imaginile de containere pe Docker Hub la fiecare modificare a codului sursa.

2 Arhitectura Sistemului (Componente si Container)

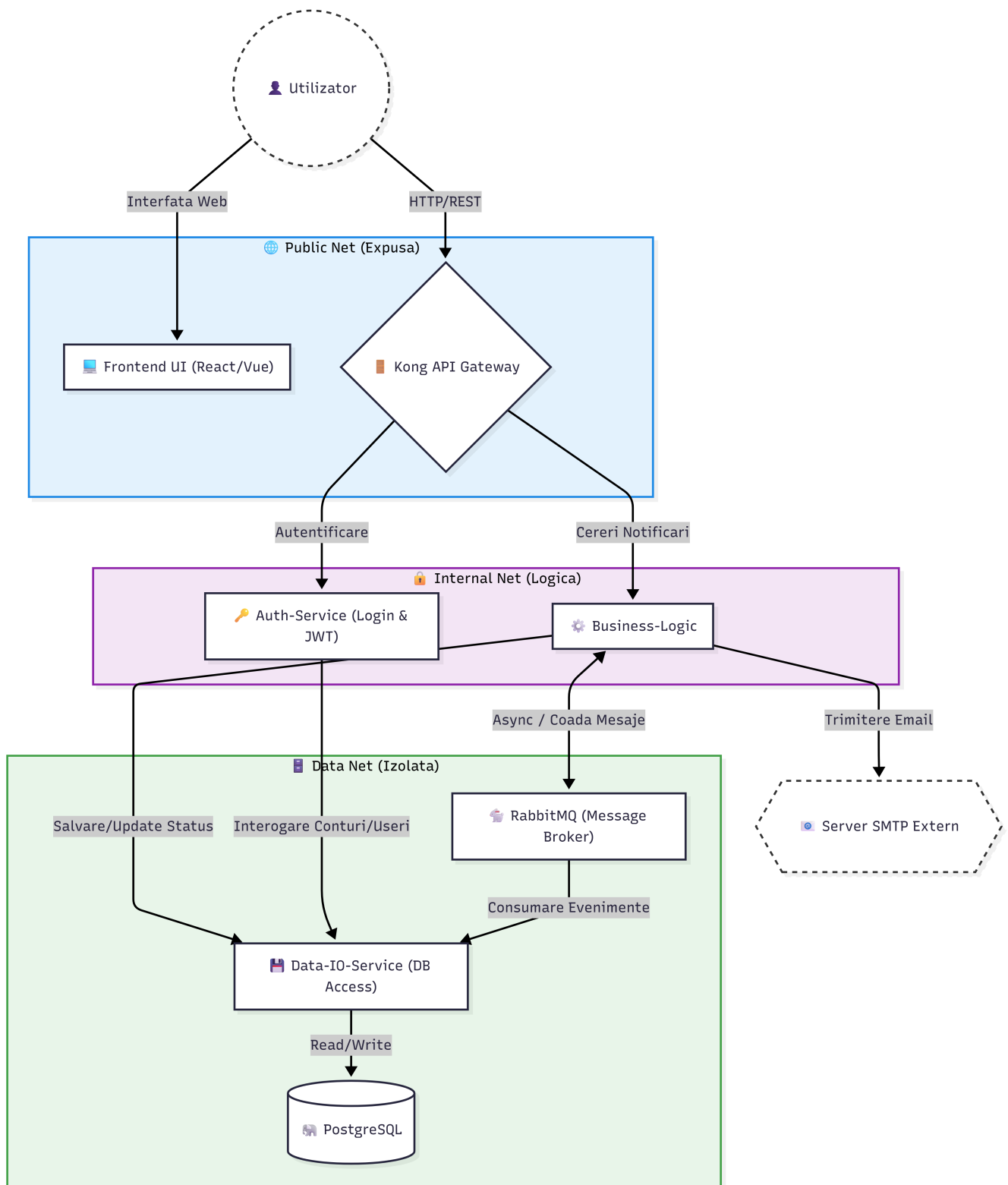


Figura 1: Diagrama de Arhitectura a Sistemului Remailer

Sistemul este alcătuit din containere izolate, împartite în două categorii principale:

2.1 A. Microservicii Proprii

- **frontend-ui:** Interfata grafica web prin care utilizatorii interactioneaza cu sistemul pentru a programa notificari.
- **auth-service:** Microserviciu responsabil cu procesul de autentificare, autorizare si generarea token-urilor de acces (JWT).
- **dispatcher-service (Business Logic):** Acesta preia cererile de la utilizatori, proceseaza logica de programare a e-mailurilor si le plaseaza in coada de mesaje.
- **data-io-service:** Microserviciu dedicat interactiunii exclusive cu baza de date (Data Access Layer). Actualizeaza starea notificarilor (*PENDING*, *SENT*) si decupleaza accesul la date de restul aplicatiei.

2.2 B. Infrastructura si Componente Suport

- **postgres-db:** Baza de date relationala utilizata pentru persistenta datelor.
- **adminer:** Utilitar web integrat pentru gestiunea vizuala a bazei de date.
- **kong-gateway:** API Gateway utilizat pentru a expune public aplicatia, asigurand rutarea traficului extern catre microserviciile din reseaua interna.
- **rabbitmq:** Sistemul de gestiune al mesajelor (*Message Broker*) care faciliteaza comunicarea asincrona si preluarea sarcinilor de expediere a e-mailurilor.
- **portainer:** Interfata vizuala utilizata pentru managementul clusterului Kubernetes.
- **prometheus & grafana:** Sistem integrat de logging si monitorizare, cuplat cu un dashboard interactiv pentru a asigura observabilitatea aplicatiei.

3 Modul de Comunicare si Topologia Retelelor

Comunicatia inter-servicii se realizeaza utilizand o abordare hibrida:

- **Sincron (HTTP/REST):** Utilizat de Kong API Gateway pentru a redirectiona traficul de la interfata web catre **auth-service** si **dispatcher-service**.
- **Asincron (Cozi de Mesaje):** Utilizat intre **dispatcher-service** si **data-io-service** prin intermediul RabbitMQ, pentru a nu bloca executia la volume mari de cereri.

Pentru a respecta standardele de securitate, serviciile sunt izolate logic in trei retele virtuale distincte:

1. **public-net:** Reteaua expusa public, continand doar **kong-gateway** si **frontend-ui**.
2. **internal-net:** Accesibila doar prin Gateway, continand **auth-service** si **dispatcher-service**.
3. **data-net:** Retea complet izolata de exterior, continand baza de date, brokerul de mesaje si **data-io-service**.

4 Stabilirea Responsabilitatilor in Cadrul Proiectului

Sarcinile au fost impartite in felul urmator:

4.1 Peter Arthur-Cristian

- ✓ Dezvoltarea microserviciului de procesare a deciziilor (`dispatcher-service` / `business-logic`).
- ✓ Dezvoltarea microserviciului pentru interactiunea exclusiva cu baza de date (`data-io-service`).
- ✓ Proiectarea schemei si configurarea bazei de date relationale (`postgres-db`).
- ✓ Implementarea si integrarea comunicarii asincrone intre servicii utilizand `rabbitmq`.
- ✓ Crearea contului de Docker Hub, a repository-urilor si automatizarea procesului de deployment (CI/CD).

4.2 Lemnaru Mihai-Daniel

- ✓ Dezvoltarea interfetei web (`frontend-ui`) si a serviciului de autentificare (`auth-service`).
- ✓ Configurarea API Gateway-ului (`kong-gateway`) pentru expunerea publica si ruta-rea sigura.
- ✓ Redactarea fisierelor de configurare Kubernetes (`.yaml`) aferente componentelor proprii si definirea topologiei de retea.
- ✓ Implementarea sistemului de monitorizare (`prometheus` & `grafana`) si a utilitarelor de management (`portainer`, `adminer`).

5 Repository-ul Proiectului

Codul sursa al aplicatiei, impreuna cu fisierele de configurare pentru infrastructura, pot fi accesate la urmatorul link:

- https://github.com/arthurpeter/Proiect_IDP